SENAI CIMATEC

Programa QuIIN

Problem-Based Learning 03 (PBL03)

Processador RISC-V de ISA Limitada em SystemVerilog

Desenvolvimento, validação com o jogo Genius e verificação em UVM

Relatório da Entrega 3 — Frentes 1 e 2

Marina de Oliveira Alencar

Outubro de 2026

Resumo

Este relatório apresenta as duas frentes do PBL03. A Frente 1 é o desenvolvimento de um processador RISC-V de ciclo único com conjunto de instruções limitado, descrito em SystemVerilog e baseado na microarquitetura do Capítulo 7 de Harris e Harris. O processador implementa 22 instruções, das quais seis são extensões além do que o capítulo apresenta, necessárias para executar o jogo Genius desenvolvido em assembly no PBL02. A validação foi feita no Vivado Simulator 2024.1: um programa de sanidade conferiu os registradores no `ecall` e o Genius foi executado sem nenhuma alteração no código em cinco cenários, que cobrem os modos "Siga" e "Mando eu", três dificuldades e duas velocidades. Em todos os cenários a pontuação final e a sequência de cores coincidiram com o esperado (`5 cenarios passaram, 0 falharam`). Os diagramas gerados pelo Vivado mostram a divisão em controller e datapath do livro. A Frente 2 verificou, em UVM, o processador de outra equipe (19 instruções): o ambiente, montado a partir do `verif-env-template` e do tutorial UVM do ChipVerify, usa um modelo de referência (ISS), um scoreboard e cobertura funcional. O processador coincidiu com o modelo em 4 programas dirigidos e 1050 programas aleatórios (cobertura de 100%), e o teste de instruções ilegais encontrou um defeito: o `mul` é executado como `add`, porque o decodificador examina só o bit 30 da instrução. O achado foi confirmado por mutação do RTL, em simulador diferente do UVM.

Palavras-chave: RISC-V; SystemVerilog; processador de ciclo único; Vivado; Genius.

Sumário

1 Introdução **1**
1.1 Contexto e objetivo 1
1.2 Escopo 1
1.3 Organização do documento 1

2 Referência: o que do Capítulo 7 foi seguido **2**
2.1 Extensões além do capítulo 3

3 Arquitetura implementada **4**

4 Passo a passo do desenvolvimento **5**

5 Validação com o testbench de sanidade **7**

6 Genius rodando no processador **8**
6.1 Montagem do sistema 9
6.2 Testbench e cenários 9
6.3 Resultados no Vivado 10

7 Diagramas gerados pelo Vivado **15**
7.1 Como os diagramas foram obtidos 16

8 Frente 2: verificação em UVM do processador da outra equipe **16**
8.1 O processador verificado 17
8.2 Arquitetura do ambiente 17
8.3 Testes 19
8.4 Cobertura 19
8.5 Execução e resultados no processador do colega 20
8.6 Achado: o `mul` é executado como `add` 20
8.7 Validação do próprio ambiente 21
8.8 Limitações da Frente 2 22

9 Conclusão e próximos passos **22**
9.1 Limitações 23
9.2 Próximos passos 23

A Como reproduzir **24**

B Como reproduzir a Frente 2 (UVM) **25**

C Código do SoC de teste do Genius **26**

Lista de Figuras

1 Módulo `riscvsingle` no Vivado (*elaborated design*). O controller (`c`) recebe `op`, `funct3`, `funct7b5` e as flags `zero`, `lt` e `ltu`; o datapath (`dp`) devolve `PC`, `ALUResult` e `WriteData`. 5
2 Ondas do teste de sanidade no Vivado Simulator (`xsim -gui`): `clk`, `reset`, `WriteData`, `DataAdr`, `MemWrite`, `ECall`, `PC` e `Instr`. 7
3 Console do `xsim`: valores dos registradores no `ecall`, todos iguais aos esperados. 7
4 Console completo do teste de sanidade: `PC` e instrução a cada ciclo, escritas na memória, valores dos registradores no `ecall` e fim da simulação em 205 ns. 8
5 Ondas da simulação inteira: os cinco pulsos de `ecall` separam os cinco cenários. 10
6 Cenário 1 no console: `PASSOU`, pontuação 36. 11
7 Cenário 1 nas ondas (0 a cerca de 1.000 µs): sequência de LEDs crescendo até o `ecall`. 11
8 Fim do cenário 1 (780 a 830 µs): pulso de `ecall` em 821 µs. 11
9 Cenário 2 no console: LEDs mostrados, toques do jogador, `ECALL` e `PASSOU`. 12
10 Cenário 2 nas ondas (800 a 900 µs): sequência 4; 4, 2; 4, 2, 4 e o `ecall` em 898 µs. 12
11 Cenário 3 no console: `PASSOU`, pontuação 544, `s3` = 16. 12
12 Cenário 3 nas ondas (cerca de 700 a 2.850 µs): partida longa entre dois pulsos de `ecall`. 13
13 Cenário 4 no console: `PASSOU`, pontuação 3, `s3` = 32. 13
14 Cenário 4 nas ondas (2.630 a 2.725 µs): rodadas 1 e 2 e o `ecall` em 2.706 µs. 13
15 Cenário 5 no console e resultado final: 5 cenários passaram, 0 falharam. 14
16 Cenário 5 nas ondas (2.690 a 2.870 µs): sequência 1; 1, 8; 1, 8, 2 e o `ecall` final. 14
17 O `PC` muda a cada ciclo de clock na escala de nanossegundos (próximo de 895 µs). 15
18 `genius_soc`: processador (`rv`), memórias (`imem1`, `dmem1`) e periféricos do Genius (`pp`). 15
19 O módulo `rv` (`riscvsingle`) aberto dentro do SoC. 16
20 Arquitetura do testbench UVM. O ISS recebe uma cópia do programa e prevê o resultado; o monitor observa o DUT; o scoreboard compara os dois. 17

Lista de Tabelas

1 Correspondência entre o Capítulo 7 e o projeto. 3
2 Extensões ao processador do Capítulo 7. 4
3 Arquivos do RTL do processador. 5
4 ISA implementada (22 instruções). 6
5 Resultado do teste de sanidade no `ecall`. 7
6 Mapa de memória do SoC de teste. 9
7 Cenários simulados e resultados. 9
8 Do tutorial UVM ao ambiente. 18
9 Os três testes do ambiente. 19
10 Resultados no processador do colega (xsim 2024.1). 20
11 Dado escrito por `sw` após `mul` e `div` com $x1 = 6$ e $x2 = 7$. 21

1 Introdução

1.1 Contexto e objetivo

O PBL03 dá continuidade aos dois projetos anteriores da trilha: no PBL01, o jogo Genius foi implementado como máquina de estados em hardware (RTL); no PBL02, o mesmo jogo foi reescrito em assembly RISC-V e validado no simulador Ripes. No PBL03, o jogo passa a rodar em um processador projetado pela própria equipe, e o processador de outra equipe é verificado com a metodologia UVM. O trabalho segue a estrutura de projeto do repositório `verif-env-template` [2], com Vivado como ferramenta de simulação.

Este relatório cobre as duas frentes: a **Frente 1**, o desenvolvimento do processador, e a **Frente 2**, a verificação em UVM do processador da outra equipe. O objetivo é construir, em SystemVerilog, um processador RISC-V de ciclo único com ISA limitada, seguindo a microarquitetura do Capítulo 7 de Harris e Harris [1], e demonstrar que ele executa o programa Genius do PBL02 sem alterações no código. Na Frente 2, o objetivo é montar um ambiente de verificação em UVM, aplicá-lo ao processador da outra equipe e relatar o que ele encontra.

1.2 Escopo

O que o relatório cobre:

- o processador de ciclo único (controller e datapath), com 22 instruções;
- a comparação, ponto a ponto, com o que o Capítulo 7 apresenta, e a justificativa de cada extensão;
- a validação com um programa de sanidade;
- a execução do Genius em simulação, com periféricos modelados em RTL;
- os diagramas gerados pelo Vivado;
- o ambiente UVM da Frente 2, seus testes, a cobertura e o defeito encontrado no processador verificado;
- o passo a passo para reproduzir tudo.

O que ficou fora, e está registrado na Seção 9: a síntese e o teste em placa, e a repetição no xsim da confirmação do defeito encontrado na Frente 2.

1.3 Organização do documento

A Seção 2 compara o projeto com o Capítulo 7. A Seção 3 descreve a arquitetura e os arquivos do RTL. A Seção 4 registra o passo a passo do desenvolvimento e a ISA

implementada. A Seção 5 mostra a validação básica, e a Seção 6 a execução do Genius. A Seção 7 apresenta os diagramas do Vivado, e a Seção 8 a verificação em UVM (Frente 2). A Seção 9 traz a conclusão, as limitações e os próximos passos. Os Apêndices A, B e C reúnem os comandos de reprodução (Frentes 1 e 2) e o código do SoC de teste.

2 Referência: o que do Capítulo 7 foi seguido

A base do projeto é o processador de ciclo único do Capítulo 7 de *Digital Design and Computer Architecture: RISC-V Edition* [1]. O capítulo constrói o datapath e o controle para `lw`, `sw`, as instruções tipo R (`add`, `sub`, `and`, `or`) e `beq`, e depois o estende com `addi` e `jal`. A organização em `riscvsingle` (controller e datapath), com memórias de instrução e de dados separadas, foi mantida. O que o programa Genius exigiu além disso foi acrescentado como extensão. A Tabela 1 mostra, elemento por elemento, o que foi seguido e o que foi estendido.

Tabela 1: Correspondência entre o Capítulo 7 e o projeto.

Elemento do Capítulo 7	**No projeto**	**Situação**
Ciclo único: uma instrução por ciclo, sem estado intermediário	`riscvsingle` = `controller` + `datapath`	Seguido
Banco de 32 registradores, duas leituras e uma escrita, `x0` fixo em 0	`regfile.sv`	Seguido
Unidade de extensão do imediato (formatos I, S, B, J)	`extend.sv`	Seguido; ganhou o formato U (`lui`/`auipc`)
ALU com `ALUControl`	`alu.sv`	Seguido; ganhou as flags `lt` (com sinal) e `ltu` (sem sinal)
Main Decoder e ALU Decoder em dois níveis, com `ALUOp`	`maindec.sv`, `aludec.sv`	Seguido; ganhou `ALUSrcA`, `Jalr` e `ecall`
`PCSrc`: PC+4 ou alvo de desvio	`controller.sv`	Estendido: mux de 3 vias, incluindo `ALUResult` (`jalr`)
`ResultSrc`: ALU, memória ou PC+4 (`jal`)	`datapath.sv`	Estendido: 4 vias, incluindo o imediato (`lui`)
Memória de dados com acesso por palavra (`lw`/`sw`)	`dmem.sv`	Estendido: acesso a byte (`lb`/`sb`) por `funct3`
`beq`, `jal` e `addi` do capítulo	Todos implementados	Seguido

2.1 Extensões além do capítulo

O assembly do Genius usa instruções que o livro não cobre, o que levou às seis extensões da Tabela 2. A ISA completa segue o formato definido pela especificação RISC-V [3].

Tabela 2: Extensões ao processador do Capítulo 7.

Extensão	**Por quê**	**O que mudou no hardware**
`lb` / `sb`	O jogo acessa vetores de bytes (sequência e cores)	`dmem.sv` seleciona a largura por `funct3` e faz extensão de sinal no `lb`
`bne` / `bge` / `bltu`	O livro só ensina `beq`; o jogo precisa de diferença, maior ou igual (com sinal) e menor (sem sinal)	Flags `lt` e `ltu` na ALU; `controller.sv` combina `funct3` com as flags em `branch_taken`
`jalr`	Retorno de sub-rotina por endereço em registrador	`PCSrc` passa a 3 vias, incluindo o `ALUResult`
`lui`	O imediato de 12 bits do `addi` não alcança os endereços do jogo	Formato U no `extend.sv`; `ResultSrc` com 4 opções
`auipc`	Constrói endereços relativos ao PC (usado em `la`)	Mux `ALUSrcA` (registrador ou PC) na entrada da ALU
`ecall`	Ponto de parada controlado, para o testbench encerrar a simulação	Sinal `ECall` dedicado, levantado pelo controller

3 Arquitetura implementada

Como no Capítulo 7, o processador é o módulo `riscvsingle`, formado por duas partes: o **controller**, que decodifica a instrução e gera os sinais de controle, e o **datapath**, que executa a instrução. As memórias ficam fora, no módulo de topo `riscv_single_top`.

O fluxo de uma instrução segue o livro. O `PC` endereça a memória de instruções e a instrução chega ao controller (`op`, `funct3`, `funct7b5`) e ao datapath. O banco de registradores lê `rs1` e `rs2`, a unidade de extensão gera o imediato conforme `ImmSrc`, e a ALU calcula com a entrada B escolhida por `ALUSrcB` (e, no `auipc`, a entrada A por `ALUSrcA`). A memória de dados é lida ou escrita por `lw`/`lb` e `sw`/`sb`. O resultado volta ao banco de registradores, escolhido por `ResultSrc` (ALU, memória, PC+4 ou imediato), e o próximo `PC` vem de `PCSrc`: PC+4, alvo de desvio ou `ALUResult` no `jalr`.

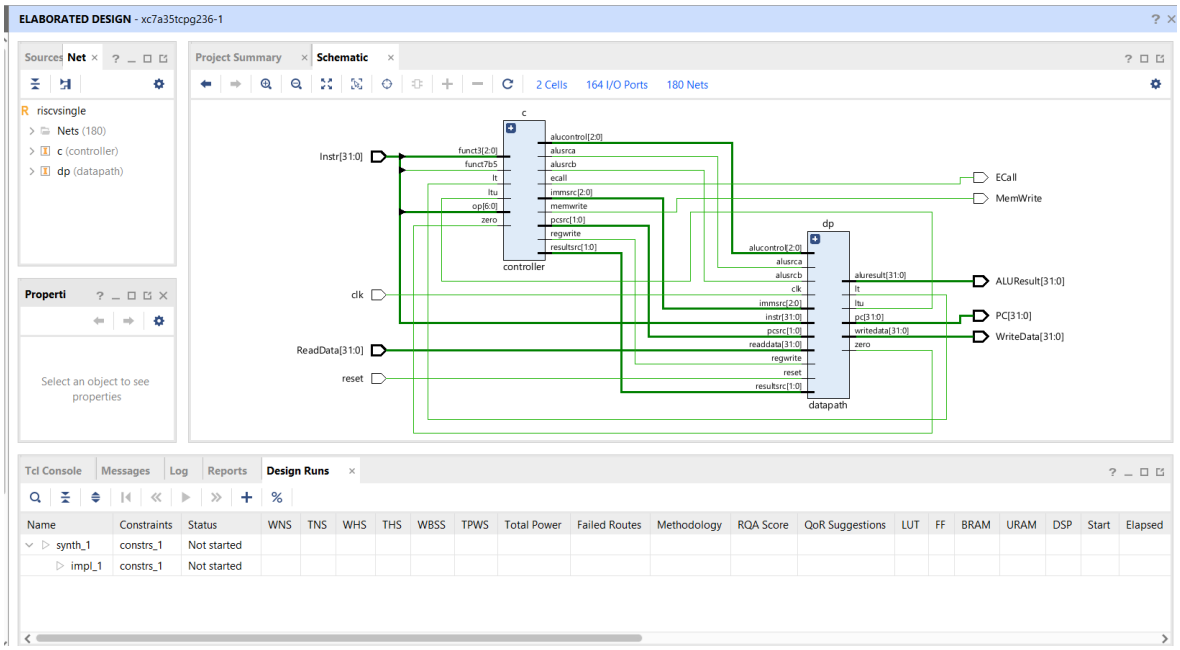


Figura 1: Módulo `riscvsingle` no Vivado (*elaborated design*). O controller (`c`) recebe `op`, `funct3`, `funct7b5` e as flags `zero`, `lt` e `ltu`; o datapath (`dp`) devolve `PC`, `ALUResult` e `WriteData`.

A Tabela 3 lista os arquivos do RTL e a função de cada um.

Tabela 3: Arquivos do RTL do processador.

Arquivo	Função
`rtl/riscvsingle.sv`	Une controller e datapath
`rtl/controller.sv`	Liga o Main Decoder e o ALU Decoder; decide `PCSrc` e `branch_taken`
`rtl/maindec.sv`	Gera os sinais de controle a partir do `opcode`
`rtl/aludec.sv`	Gera `ALUControl` a partir de `ALUOp`, `funct3` e `funct7b5`
`rtl/datapath.sv`	PC, banco de registradores, extensão, ALU e multiplexadores
`rtl/alu.sv`	Operações aritméticas e lógicas; flags `zero`, `lt`, `ltu`
`rtl/regfile.sv`	32 registradores de 32 bits
`rtl/extend.sv`	Imediatos dos formatos I, S, B, J e U
`rtl/imem.sv`, `rtl/dmem.sv`	Memórias de instrução e de dados (byte e palavra)
`rtl/riscv_single_top.sv`	Processador mais `imem` e `dmem`, usado no teste de sanidade

4 Passo a passo do desenvolvimento

A ordem abaixo é a que foi seguida, do estudo do Capítulo 7 até o jogo rodando.

1. **Levantar as instruções necessárias.** Com o assembly do Genius (PBL02) em mãos, foram listadas todas as instruções usadas. Parte delas já está no Capítulo 7; o resto virou extensão (Seção 2).
2. **Definir a ISA limitada.** Ficaram 22 instruções, listadas na Tabela 4.
3. **Implementar o núcleo do Capítulo 7.** Banco de registradores, extensão do imediato, ALU, Main Decoder, ALU Decoder, controller e datapath, nesta ordem, cada um como módulo separado.
4. **Acrescentar as extensões.** `lb`/`sb` na memória de dados; flags `lt`/`ltu` para `bne`/`bge`/`bltu`; `PCSrc` de 3 vias para `jalr`; formato U e `ResultSrc` de 4 vias para `lui`; `ALUSrcA` para `auipc`; sinal `ECall`.
5. **Montar o topo e as memórias.** `riscv_single_top` liga o processador a `imem` e `dmem`; o programa é carregado por `$readmemh`.
6. **Escrever o teste de sanidade.** Um programa curto (`sim/encode.py` gera o `program.hex`) exercita as classes de instrução, e `tb/testbench.sv` confere os registradores no `ecall` (Seção 5).
7. **Validar no fluxo oficial.** Checagem rápida com Icarus Verilog e depois execução no Vivado (`xvlog`, `xelab`, `xsim`) pelo `verif-env-template`, com CMake [2, 6].
8. **Rodar o Genius.** Um SoC de teste liga o núcleo a periféricos mapeados em memória, e um testbench joga o jogo e confere o resultado (Seção 6).
9. **Gerar os diagramas.** *Elaborated design* do Vivado, para mostrar que o RTL corresponde ao desenho do capítulo (Seção 7).

Tabela 4: ISA implementada (22 instruções).

Instrução	Tipo	Opcode	Função
`add`, `sub`, `and`, `or`, `xor`	R	`0110011`	Operações entre registradores
`addi`, `andi`, `slli`, `srli`	I	`0010011`	Operações com imediato
`lw`, `lb`	I	`0000011`	Carrega palavra / byte (com extensão de sinal)
`sw`, `sb`	S	`0100011`	Armazena palavra / byte
`beq`, `bne`, `bge`, `bltu`	B	`1100011`	Desvios condicionais
`jal`	J	`1101111`	Salto com retorno
`jalr`	I	`1100111`	Salto indireto com retorno
`lui`	U	`0110111`	Imediato nos bits altos
`auipc`	U	`0010111`	PC mais imediato nos bits altos
`ecall`	I	`1110011`	Marcador de fim da simulação

5 Validação com o testbench de sanidade

Antes do Genius, um programa curto exercita uma instrução de cada classe: aritmética, `sw`/`lw`, `sb`/`lb`, um laço com `bltu`, `auipc` e o `ecall` final. O `tb/testbench.sv` instancia `riscv_single_top`, gera o clock e, ao ver o `ecall`, confere os registradores. A Tabela 5 mostra o resultado.

Tabela 5: Resultado do teste de sanidade no `ecall`.

Registrador	Papel no teste	Esperado	Obtido
`x28` (`t3`)	lido de volta da memória (`sw` + `lw`)	15	15
`x30` (`t5`)	lido de volta da memória (`sb` + `lb`)	65	65
`x12` (`a2`)	`auipc` no endereço `0x34`	`0x34`	`0x34`
`x10` (`a0`)	contador do laço (`bltu`)	3	3

A simulação termina no `$finish` em 205 ns. A Figura 2 mostra as ondas do Vivado Simulator para o programa inteiro, e a Figura 3 o console com os valores finais.

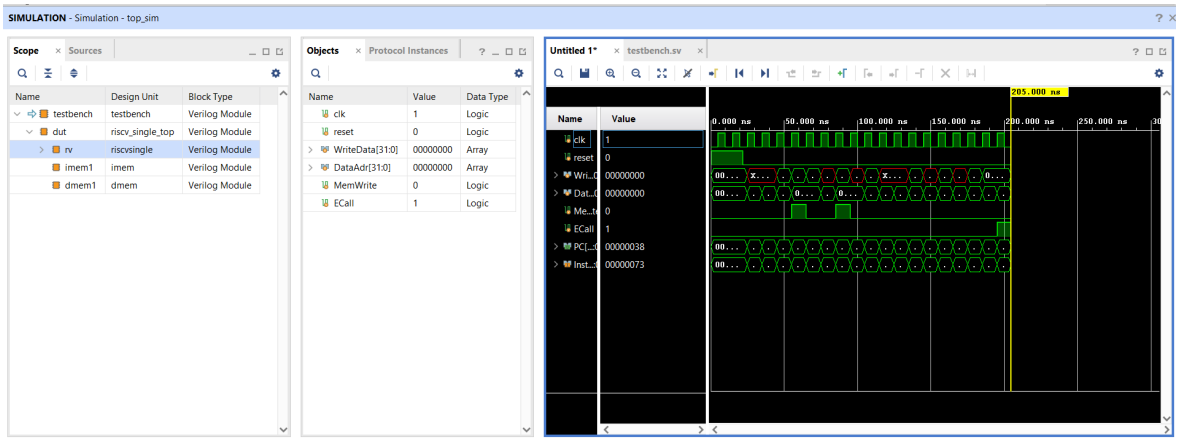


Figura 2: Ondas do teste de sanidade no Vivado Simulator (`xsim -gui`): `clk`, `reset`, `WriteData`, `DataAdr`, `MemWrite`, `ECall`, `PC` e `Instr`.

```
---- ECALL em PC=00000038 ----
a0 = 3 (0x00000003)
t3 (mem[0x2000]) esperado=15 -> x28 = 15
t5 (mem[0x2004]) esperado=65 -> x30 = 65
a2 (auipc)       esperado=0x34 -> x12 = 00000034
$finish called at time : 205 ns : File "C:/Users/marin/Documents/pbl03/tb/testbench.sv" Line 44
```

Figura 3: Console do `xsim`: valores dos registradores no `ecall`, todos iguais aos esperados.

A Figura 4 mostra o console completo da mesma execução, na máquina de desenvolvimento (Vivado 2024.1). A cada ciclo de clock (10 ns) o testbench imprime o `PC` e a instrução em execução, e, quando há escrita, o endereço e o dado da memória:

`MEM[00002000] <= 0000000f` (o `sw`) e `MEM[00002004] <= 00000041` (o `sb`). O trecho de `PC=0000002c` a `PC=00000030`, que se repete três vezes, é o laço com `bltu`; ele termina quando o `a0` chega a 3, e o programa segue até o `ecall` em `PC=00000038`.

```
marin@Marina MINGW64 ~/Documents/pbl03 (main)
$ cd build

marin@Marina MINGW64 ~/Documents/pbl03/build (main)
$ cmd //c "xsim top_sim -R"

****** xsim v2024.1 (64-bit)
  **** SW Build 5076996 on Wed May 22 18:37:14 MDT 2024
  **** IP Build 5075265 on Wed May 22 21:45:21 MDT 2024
  **** SharedData Build 5076995 on Wed May 22 18:29:18 MDT 2024
  **** Start of session at: Wed Oct 14 15:17:43 2026
    ** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
    ** Copyright 2022-2024 Advanced Micro Devices, Inc. All Rights Reserved.

source xsim.dir/top_sim/xsim_script.tcl
# xsim {top_sim} -autoloadwcfg -runall
Time resolution is 1 ps
run -all
t=25000  PC=00000000  Instr=00002437
t=35000  PC=00000004  Instr=00500293
t=45000  PC=00000008  Instr=00a00313
t=55000  PC=0000000c  Instr=006283b3
t=65000  PC=00000010  Instr=00742023
          MEM[00002000] <= 0000000f
t=75000  PC=00000014  Instr=00042e03
t=85000  PC=00000018  Instr=04100e93
t=95000  PC=0000001c  Instr=01d40223
          MEM[00002004] <= 00000041
t=105000  PC=00000020  Instr=00440f03
t=115000  PC=00000024  Instr=00000513
t=125000  PC=00000028  Instr=00300593
t=135000  PC=0000002c  Instr=00150513
t=145000  PC=00000030  Instr=feb56ee3
t=155000  PC=0000002c  Instr=00150513
t=165000  PC=00000030  Instr=feb56ee3
t=175000  PC=0000002c  Instr=00150513
t=185000  PC=00000030  Instr=feb56ee3
t=195000  PC=00000034  Instr=00000617
t=205000  PC=00000038  Instr=00000073
---- ECALL em PC=00000038 ----
a0 = 3 (0x00000003)
t3 (mem[0x2000]) esperado=15 -> x28 = 15
t5 (mem[0x2004]) esperado=65 -> x30 = 65
a2 (auipc)       esperado=0x34 -> x12 = 00000034
$finish called at time : 205 ns : File "C:/Users/marin/Documents/pbl03/tb/testbench.sv" Line 44
exit
INFO: [Common 17-206] Exiting xsim at Wed Oct 14 15:17:48 2026...

marin@Marina MINGW64 ~/Documents/pbl03/build (main)
$
```

Figura 4: Console completo do teste de sanidade: `PC` e instrução a cada ciclo, escritas na memória, valores dos registradores no `ecall` e fim da simulação em 205 ns.

A validação foi feita em dois níveis: uma checagem rápida de sintaxe e funcionalidade com Icarus Verilog durante o desenvolvimento, e a execução no fluxo oficial da disciplina, com o Vivado pelo `verif-env-template` (CMake com `xvlog`, `xelab` e `xsim`) [2, 6]. Na máquina utilizada (Windows com Git Bash), o `xsim` precisa ser chamado por `cmd //c`, porque o script de inicialização do Xilinx reclama de `Unsupported architecture: i686-pc` no Git Bash; o procedimento completo está no Apêndice A.

6 Genius rodando no processador

O programa Genius do PBL02, em assembly RISC-V, roda no processador **sem nenhuma alteração no código**. A verificação é feita somente em simulação: os periféricos do jogo são modelos em RTL, e não hardware da Basys3.

6.1 Montagem do sistema

O jogo precisa de LEDs, chaves, botões e um relógio, que o núcleo não tem. Um SoC de teste (`genius_soc.sv`) liga o núcleo às memórias e a um bloco de periféricos mapeados em memória (`genius_periph.sv`); o código está no Apêndice C. Os endereços dos periféricos entram na montagem como símbolos (`-defsym`), e um pequeno `crt0.s` inicializa o `sp`, como o Ripes faz. O programa é montado com `riscv64-unknown-elf-as -march=rv32i`, ligado com `.text` em `0x0` e `.data` em `0x2000`, e convertido em `genius.hex` (161 palavras) e `data.hex` (48 bytes). A Tabela 6 mostra o mapa de memória.

Tabela 6: Mapa de memória do SoC de teste.

Endereço	O que é	Acesso
`0x0000_0000`	Memória de instruções (`genius.hex`)	leitura
`0x0000_2000`	Dados: `seq_buffer` em `0x2000`, `cores` em `0x2020` (`data.hex`)	lb/sb
`0x0000_3FF0`	Topo da pilha (`sp`, definido pelo `crt0.s`)	lw/sw
`0xF000_0000`	LEDs: 4 LEDs de 24 bits (UP, DOWN, LEFT, RIGHT)	escrita
`0xF000_0100`	Chaves: bit 4 início, bit 3 modo, bit 2 velocidade, bits 1:0 dificuldade	leitura
`0xF000_0200`	D-Pad: 4 botões com trava, limpa quando o programa lê	leitura
`0xF000_0300`	Relógio (RTC): avança 1 ms por ciclo de clock	leitura

O desassembly confirmou que todas as instruções do Genius já estavam na ISA implementada, então o núcleo não precisou de nenhuma mudança para o jogo.

6.2 Testbench e cenários

O `tb/genius_tb.sv` joga o jogo sozinho: aperta os botões do D-Pad, observa os LEDs e compara cada cor mostrada com um modelo de referência (LFSR de 8 bits, semente `0xA5`, *taps* nos bits 7, 5, 4 e 3). Também confere a pontuação impressa no `ecall` e o maior tamanho de sequência (registrador `s3`). A Tabela 7 lista os cinco cenários.

Tabela 7: Cenários simulados e resultados.

#	Modo	Dific.	Veloc.	Situação	Pontos	`s3`
1	Siga	Fácil	Lento	Partida completa, 8 rodadas	36	8
2	Siga	Fácil	Rápido	Erro de propósito na rodada 3	6	8
3	Siga	Médio	Rápido	Partida completa, 16 rodadas	544	16
4	Siga	Difícil	Lento	Erro de propósito na rodada 2	3	32
5	Mando eu	Fácil	Lento	UP, RIGHT, DOWN; erro na rodada 3	3	8

Nos cinco cenários a pontuação foi igual à esperada, o `s3` bateu e as divergências na sequência de cores foram zero.

6.3 Resultados no Vivado

Os resultados foram obtidos em uma única simulação no `xsim`, em que os cinco cenários rodam em sequência. Para cada cenário há dois registros: o console do Vivado (Tcl Console), que mostra o que o testbench observou e o veredito, e as formas de onda dos sinais `led_prev`, `dpad_btn`, `score` e `ecall`. No console, cada linha `LED n acende` traz o índice do LED (0 = UP, 1 = DOWN, 2 = LEFT, 3 = RIGHT), e `jogador aperta` traz o botão pressionado. Nas ondas, o `led_prev` tem um bit por LED: 1 = UP, 2 = DOWN, 4 = LEFT e 8 = RIGHT.

Na onda, o `score` aparece como -1 (decimal com sinal) enquanto a partida está em andamento. A pontuação final de cada cenário, impressa no `ecall`, está na linha `pontuacao` do console.

Visão geral. A Figura 5 mostra a simulação inteira (0 a 2.845 µs). Cada pulso de `ecall` marca o fim de uma partida: em cerca de 821, 898, 2.647, 2.706 e 2.844 µs.

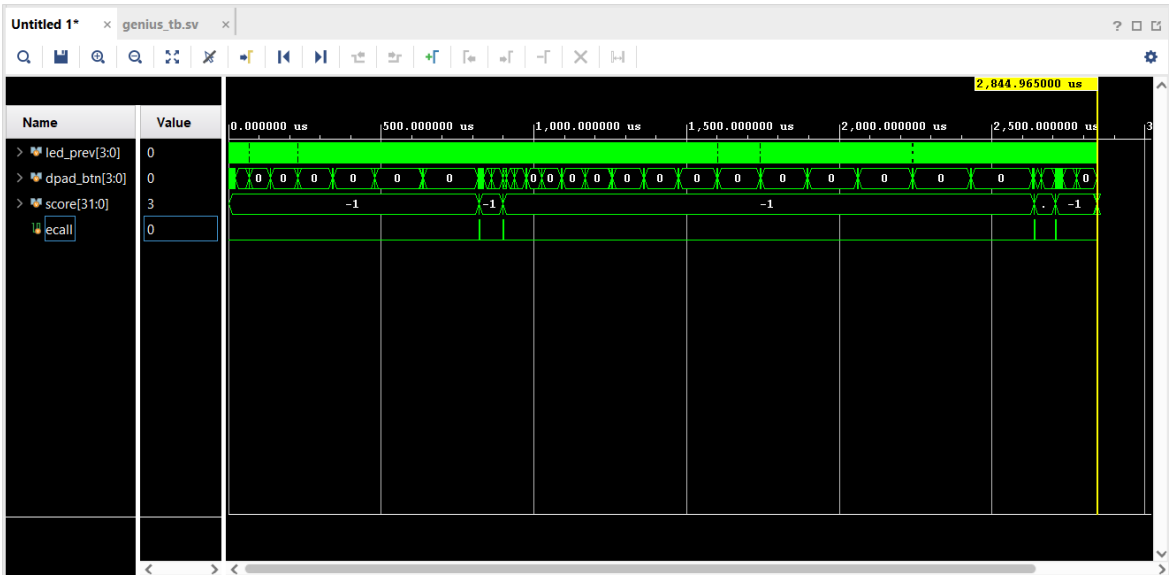


Figura 5: Ondas da simulação inteira: os cinco pulsos de `ecall` separam os cinco cenários.

Cenário 1: modo Siga, fácil, lento, partida completa. O console (Figura 6) mostra pontuação 36, `s3` igual a 8 e nenhuma divergência. Na onda (Figura 7), o `led_prev` mostra a sequência crescendo a cada rodada (4; 4, 2; 4, 2, 4; . . .), e o `ecall` em cerca de 821 µs encerra a partida. A Figura 8 detalha esse fim, com o `led_prev` voltando a 0.

```
Genius no nucleo RISC-V (PBL03) - 161 palavras de programa carregadas

=== Cenario 1: modo Siga, facil, lento, partida completa (8 rodadas) ===
  pontuacao = 36 (esperado 36) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
  >>> Cenario 1: modo Siga, facil, lento, partida completa (8 rodadas): PASSOU
```

Figura 6: Cenário 1 no console: `PASSOU`, pontuação 36.

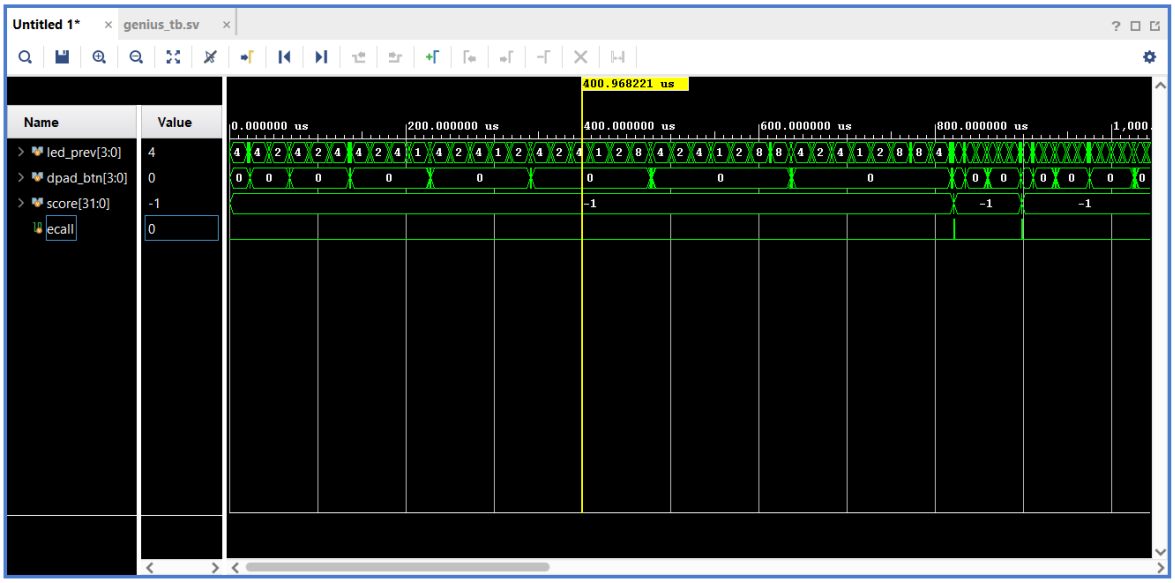


Figura 7: Cenário 1 nas ondas (0 a cerca de 1.000 µs): sequência de LEDs crescendo até o `ecall`.

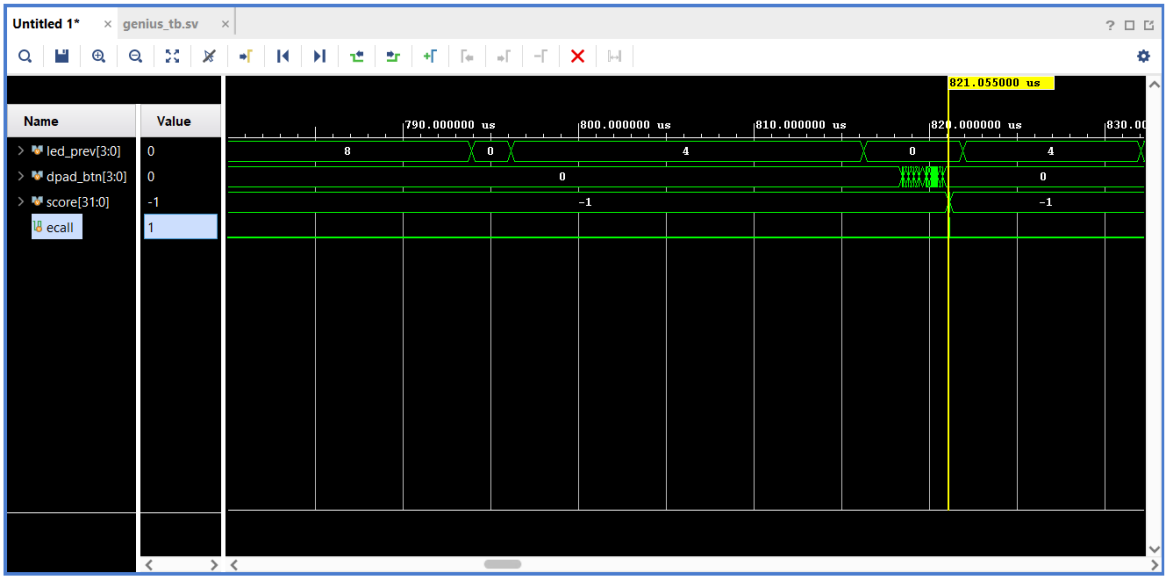


Figura 8: Fim do cenário 1 (780 a 830 µs): pulso de `ecall` em 821 µs.

Cenário 2: modo Siga, fácil, rápido, erro na rodada 3. O console (Figura 9) registra cada LED aceso e cada toque do jogador. O jogador acerta LEFT e DOWN e,

na rodada 3, aperta RIGHT em vez de LEFT, em 897 µs. O `ecall` vem em 898 µs, e a pontuação é 6. Na onda (Figura 10), o `led_prev` alterna entre 4 (LEFT) e 2 (DOWN), com o LED apagando entre as cores.

```
=== Cenario 2: modo Siga, facil, rapido, erro de proposito na rodada 3 ===
  [821000 ns] LED 2 acende: LEFT/azul
  [834000 ns] jogador aperta LEFT/azul
  [834000 ns] LED 2 acende: LEFT/azul
  [847000 ns] LED 1 acende: DOWN/verde
  [859000 ns] jogador aperta LEFT/azul
  [860000 ns] jogador aperta DOWN/verde
  [860000 ns] LED 2 acende: LEFT/azul
  [873000 ns] LED 1 acende: DOWN/verde
  [885000 ns] LED 2 acende: LEFT/azul
  [897000 ns] jogador aperta RIGHT/amarelo
  [898000 ns] ECALL: jogo terminou, pontuacao impressa = 6
  pontuacao = 6 (esperado 6) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
  >>> Cenario 2: modo Siga, facil, rapido, erro de proposito na rodada 3: PASSOU
```

Figura 9: Cenário 2 no console: LEDs mostrados, toques do jogador, `ECALL` e `PASSOU`.

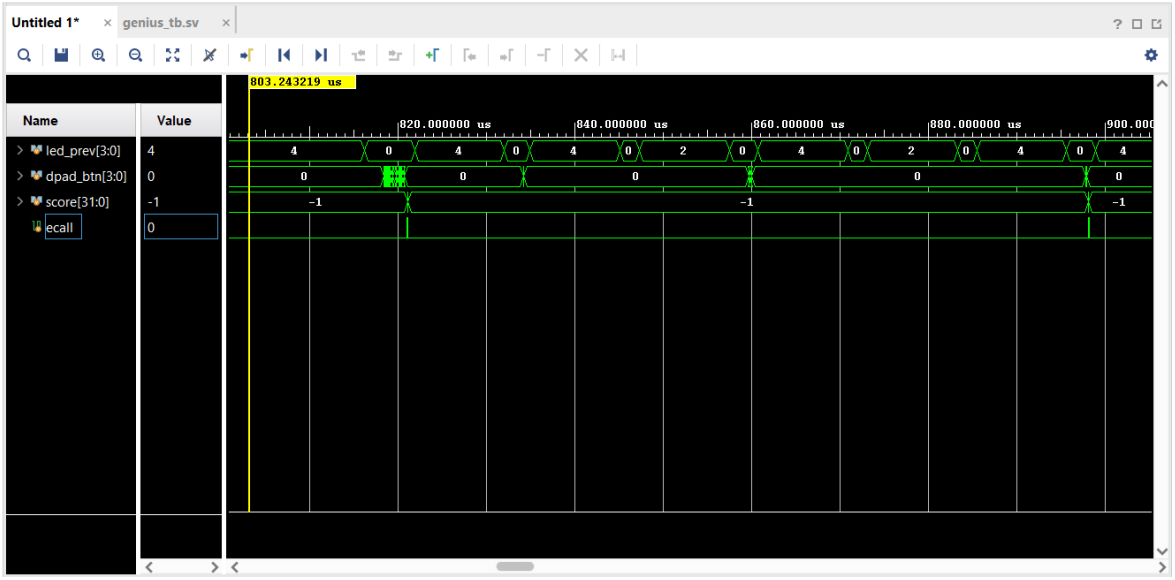


Figura 10: Cenário 2 nas ondas (800 a 900 µs): sequência 4; 4, 2; 4, 2, 4 e o `ecall` em 898 µs.

Cenário 3: modo Siga, médio, rápido, partida completa. É o cenário mais longo, com 16 rodadas, e o console (Figura 11) mostra pontuação 544 e `s3` igual a 16. Na onda (Figura 12), o `led_prev` muda sem parar entre o `ecall` do cenário 2 (cerca de 898 µs) e o do cenário 3 (cerca de 2.647 µs), o que aparece como um bloco verde contínuo.

```
=== Cenario 3: modo Siga, medio, rapido, partida completa (16 rodadas) ===
  pontuacao = 544 (esperado 544) | tamanho max s3 = 16 (esperado 16) | divergencias = 0
  >>> Cenario 3: modo Siga, medio, rapido, partida completa (16 rodadas): PASSOU
```

Figura 11: Cenário 3 no console: `PASSOU`, pontuação 544, `s3` = 16.

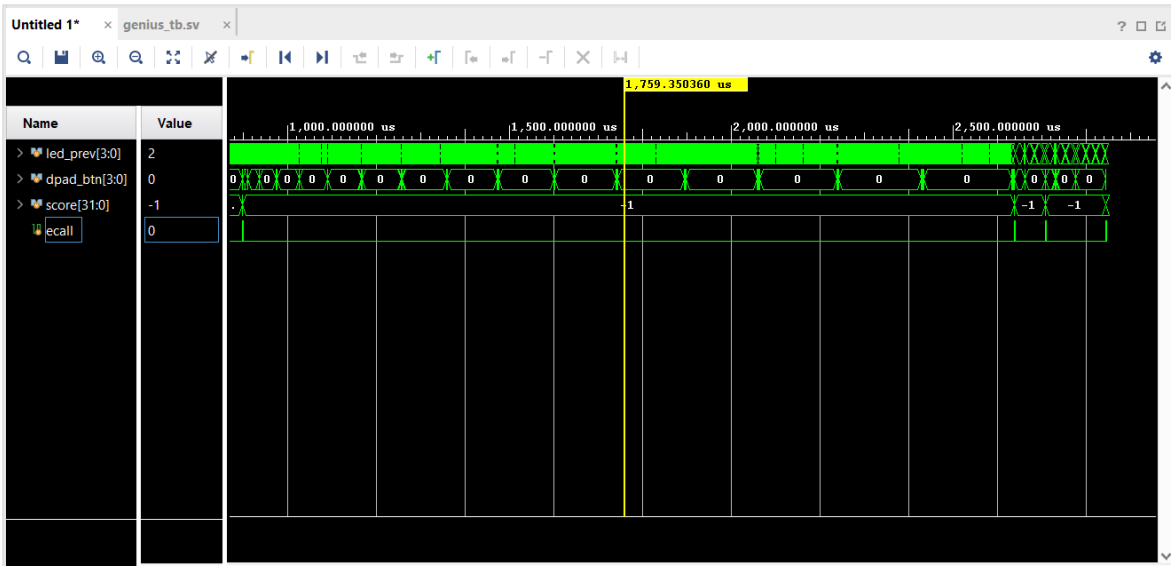


Figura 12: Cenário 3 nas ondas (cerca de 700 a 2.850 µs): partida longa entre dois pulsos de `ecall`.

Cenário 4: modo Siga, difícil, lento, erro na rodada 2. O console (Figura 13) mostra pontuação 3 e `s3` igual a 32. Na onda (Figura 14), entre os `ecall` em 2.637 e 2.706 µs, o `led_prev` mostra 4 na rodada 1 e 4, 2 na rodada 2, e a partida termina em seguida, com o erro do jogador.

```
=== Cenario 4: modo Siga, dificil, lento, erro de proposito na rodada 2 ===
  pontuacao = 3 (esperado 3) | tamanho max s3 = 32 (esperado 32) | divergencias = 0
  >>> Cenario 4: modo Siga, dificil, lento, erro de proposito na rodada 2: PASSOU
```

Figura 13: Cenário 4 no console: `PASSOU`, pontuação 3, `s3` = 32.

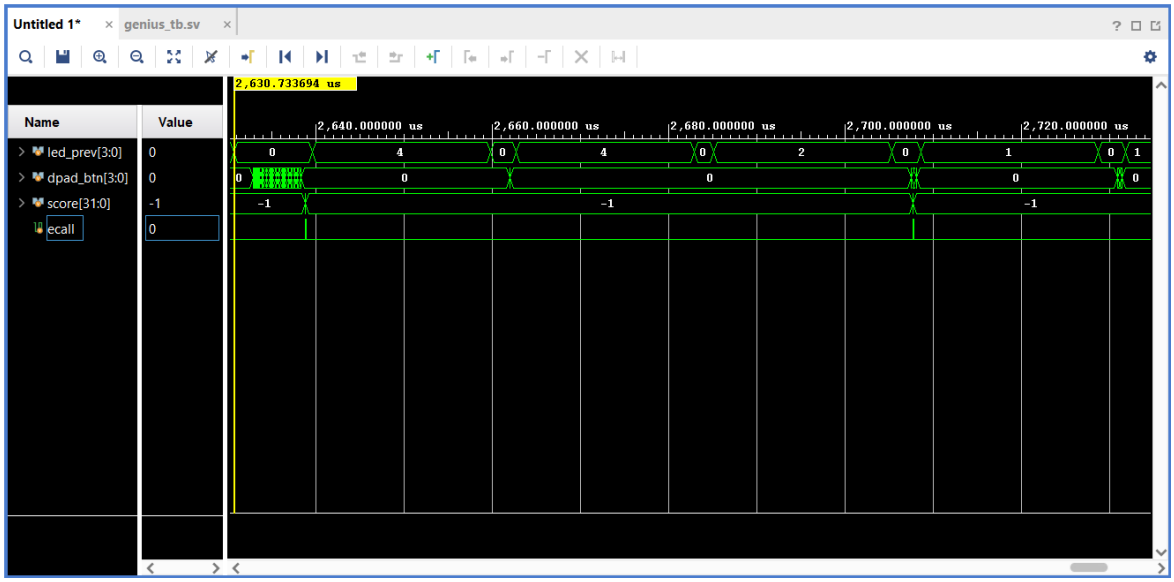


Figura 14: Cenário 4 nas ondas (2.630 a 2.725 µs): rodadas 1 e 2 e o `ecall` em 2.706 µs.

Cenário 5: modo Mando eu, fácil, lento, erro na rodada 3. Nesse modo o jogador dá a sequência e o jogo a repete. O console (Figura 15) mostra os toques UP, RIGHT e DOWN (LEDs 0, 3 e 1), a pontuação 3 e o resultado final: `5 cenarios passaram, 0 falharam`. Na onda (Figura 16), o `led_prev` mostra 1; 1, 8; 1, 8, 2, e o `ecall` final, em 2.844 µs, encerra a simulação.

```
=== Cenario 5: modo Mando eu, facil, lento, erro de proposito na rodada 3 ===
  [2708000 ns] jogador aperta UP/vermelho
  [2708000 ns] LED 0 acende: UP/vermelho
  [2730000 ns] jogador aperta UP/vermelho
  [2731000 ns] jogador aperta RIGHT/amarelo
  [2731000 ns] LED 0 acende: UP/vermelho
  [2754000 ns] LED 3 acende: RIGHT/amarelo
  [2776000 ns] jogador aperta UP/vermelho
  [2776000 ns] jogador aperta RIGHT/amarelo
  [2777000 ns] jogador aperta DOWN/verde
  [2777000 ns] LED 0 acende: UP/vermelho
  [2799000 ns] LED 3 acende: RIGHT/amarelo
  [2822000 ns] LED 1 acende: DOWN/verde
  [2844000 ns] jogador aperta DOWN/verde
  [2844000 ns] ECALL: jogo terminou, pontuacao impressa = 3
  pontuacao = 3 (esperado 3) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
  >>> Cenario 5: modo Mando eu, facil, lento, erro de proposito na rodada 3: PASSOU

============================================
 RESULTADO: 5 cenarios passaram, 0 falharam
============================================
```

Figura 15: Cenário 5 no console e resultado final: 5 cenários passaram, 0 falharam.

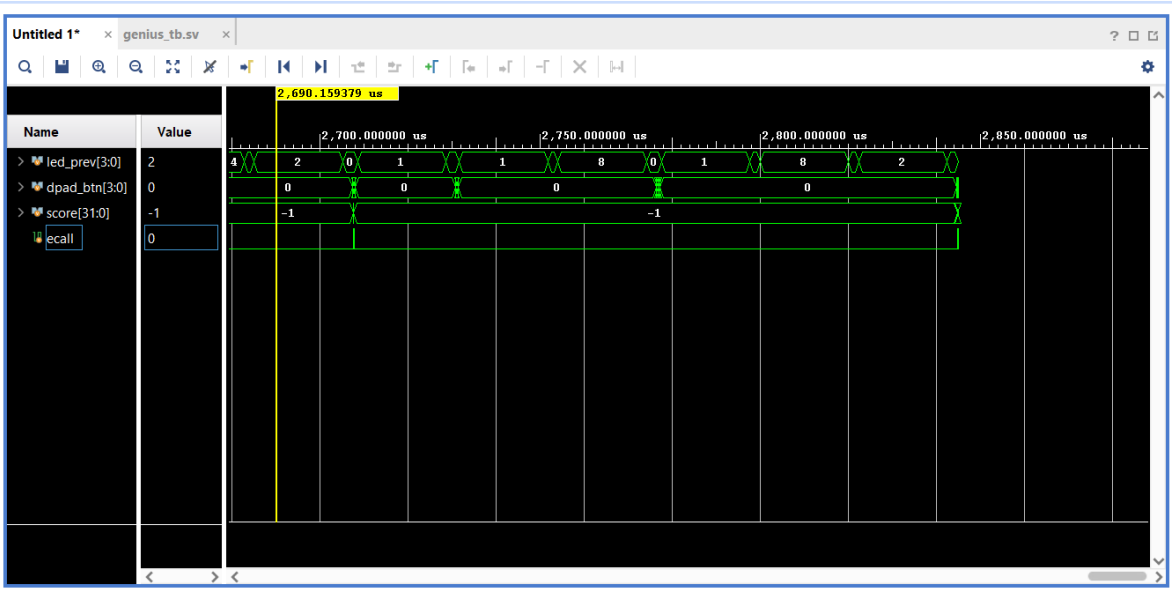


Figura 16: Cenário 5 nas ondas (2.690 a 2.870 µs): sequência 1; 1, 8; 1, 8, 2 e o `ecall` final.

Processador em execução. Com zoom em nanossegundos (Figura 17), o `PC` muda a cada ciclo de clock (10 ns), mostrando o núcleo executando as instruções do Genius.

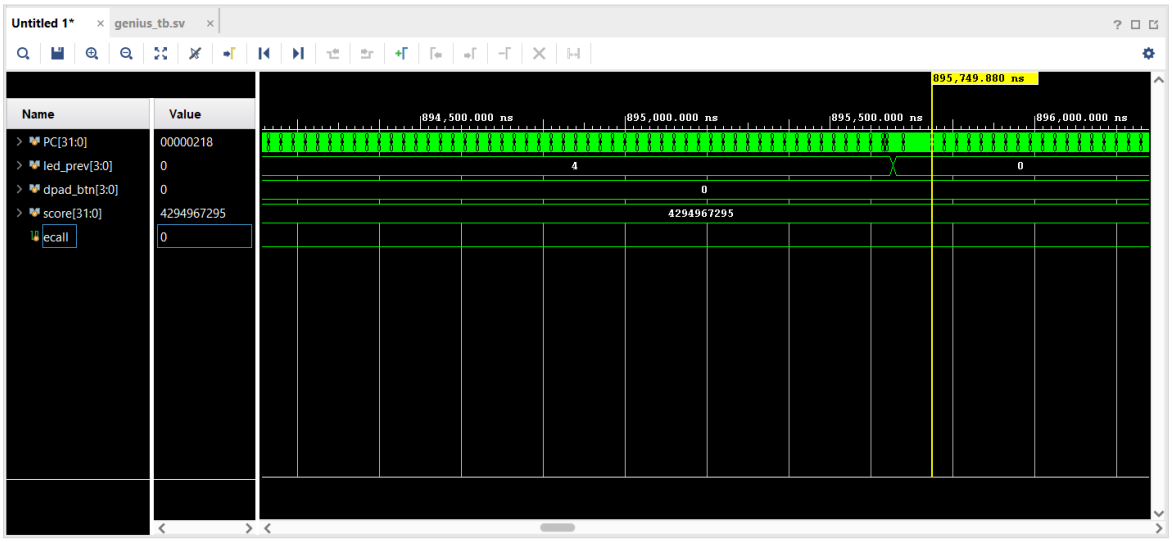


Figura 17: O `PC` muda a cada ciclo de clock na escala de nanossegundos (próximo de 895 µs).

7 Diagramas gerados pelo Vivado

O *elaborated design* do Vivado desenha o circuito a partir do próprio RTL. Ele serve como prova de que o código corresponde à microarquitetura do Capítulo 7: o `riscvsingle` aparece dividido em controller e datapath (Figura 1), como no livro.

O SoC de teste é o módulo `genius_soc` (Figura 18). Nele, `rv` é o processador, `imem1` guarda o programa Genius, `dmem1` é a memória de dados e `pp` são os periféricos do jogo. As entradas externas são `clk`, `reset`, `switches[4:0]` e `dpad_btn[3:0]`; a saída é `leds[95:0]`. O comparador `RTL_EQ` detecta endereços que começam em `0xF` (periféricos), a porta `RTL_AND` impede que essas escritas cheguem à `dmem1`, e o `RTL_MUX` escolhe se o dado lido vem da memória ou dos periféricos.

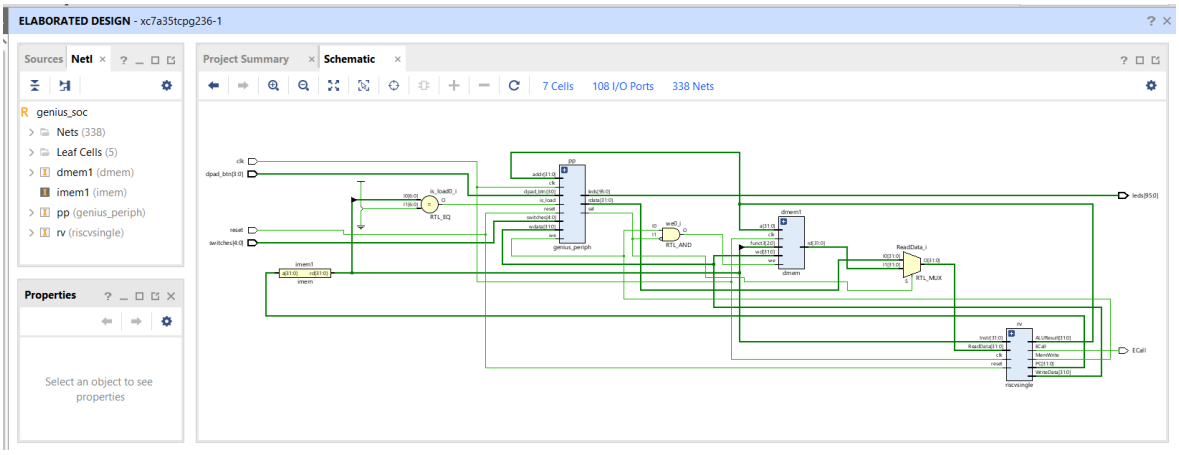


Figura 18: `genius_soc`: processador (`rv`), memórias (`imem1`, `dmem1`) e periféricos do Genius (`pp`).

A Figura 19 mostra o módulo `rv` aberto dentro do SoC, com o controller (`c`) e o

datapath (`dp`) ligados pelos sinais de controle.

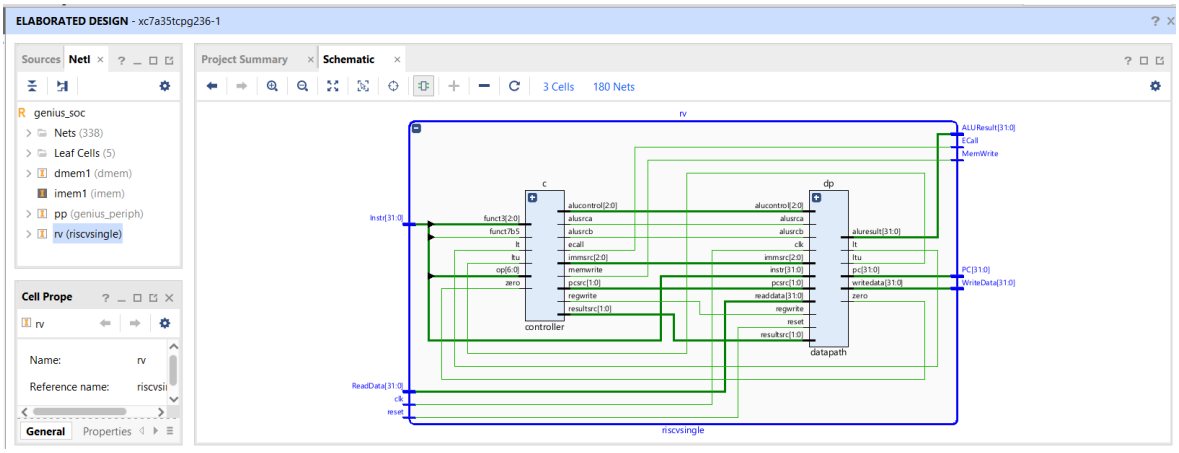


Figura 19: O módulo `rv` (`riscvsingle`) aberto dentro do SoC.

7.1 Como os diagramas foram obtidos

1. No Vivado, criar um projeto só para visualização e adicionar os fontes: `add_files [glob .../pbl03/rtl/*.sv]`.
2. Definir o módulo de topo: `set_property top riscvsingle [current_fileset]` para o processador, ou `genius_soc` para o sistema.
3. Para o `genius_soc`, aumentar o limite de elaboração das memórias (comando no Apêndice A).
4. Em *RTL ANALYSIS*, abrir *Open Elaborated Design* e depois *Schematic*. Um duplo clique em um bloco abre o nível de dentro.

O aviso do Vivado sobre não achar `genius.hex` e `data.hex` nessa etapa não afeta o diagrama; só afeta o conteúdo inicial das memórias.

8 Frente 2: verificação em UVM do processador da outra equipe

A Frente 2 do PBL03 pede a verificação, em UVM, do processador de outra equipe. O alvo é o repositório `Penelu/RISC_V-PBL` [4], um RISC-V de ciclo único em SystemVerilog com memórias de instrução e de dados próprias. O ambiente foi montado a partir do `verif-env-template` [2] e dos conceitos do tutorial UVM do ChipVerify [5], e simulado no Vivado Simulator (xsim) 2024.1.

8.1 O processador verificado

O README do colega lista 19 instruções e promete que *qualquer outra instrução é ignorada: não escreve registrador nem memória.* Essa frase é o critério de verificação da parte negativa (Seção 8.3). As 19 instruções são:

- aritméticas e lógicas: `add`, `addi`, `and`, `andi`, `xor`, `xori`, `srli`;
- memória: `lw`, `lbu`, `sw`, `sb`;
- controle de fluxo: `beq`, `bne`, `blt`, `bge`, `jal`, `jalr`;
- constantes: `lui`, `auipc`.

O processador não tem `ecall`. O fim de um programa é marcado pelo salto para si mesmo, `jal x0,0`.

8.2 Arquitetura do ambiente

O ambiente segue a arquitetura clássica do tutorial (sequencer, driver, interface, DUT, monitor e scoreboard) e acrescenta um modelo de referência e um coletor de cobertura (Figura 20). O modelo de referência é um simulador do conjunto de instruções (ISS) escrito em SystemVerilog: o driver entrega uma cópia de cada programa a ele, e ele publica, ciclo a ciclo, o que o processador deveria fazer. O scoreboard compara esse trace esperado com o trace observado pelo monitor.

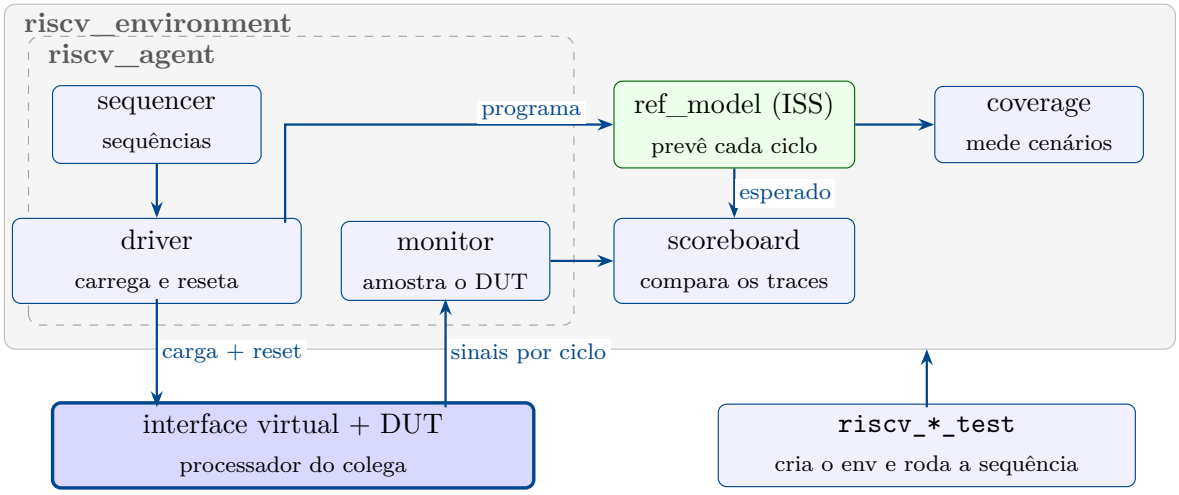


Figura 20: Arquitetura do testbench UVM. O ISS recebe uma cópia do programa e prevê o resultado; o monitor observa o DUT; o scoreboard compara os dois.

A Tabela 8 relaciona cada conceito do tutorial à classe correspondente no ambiente.

Tabela 8: Do tutorial UVM ao ambiente.

Conceito	**Classe**	**O que faz**
Transação	`riscv_program`, `riscv_trace`	Um programa inteiro; um ciclo observado
Sequencer	`riscv_sequencer`	Entrega os programas ao driver
Driver	`riscv_driver`	Carrega as memórias, reseta o DUT, espera o fim
Monitor	`riscv_monitor`	Amostra o DUT a cada ciclo (na borda de descida)
Agent	`riscv_agent`	Agrupa sequencer, driver e monitor
Modelo de referência	`riscv_ref_model`	ISS: publica o trace esperado
Scoreboard	`riscv_scoreboard`	Compara esperado e obtido, ciclo a ciclo
Cobertura	`riscv_coverage`	Mede os cenários exercitados
Env e testes	`riscv_environment`, `riscv_*_test`	Montam o ambiente; escolhem a sequência

Os demais mecanismos do tutorial aparecem assim: as quatro fases (`build`, `connect`, `run`, `report`) organizam a criação e a ligação dos componentes; o teste levanta uma *objection* na fase `run` para que a simulação não termine antes do último programa; todo objeto nasce da *factory* (`type_id::create`); a interface virtual chega ao driver e ao monitor pelo `uvm_config_db` com o nome `intf`; e as mensagens usam `uvm_info`, `uvm_warning`, `uvm_error` e `uvm_fatal`. O RAL (*register abstraction layer*) não é usado, porque o processador não tem registradores de configuração acessados por barramento.

Scoreboard. Para cada ciclo, o scoreboard compara o PC, a instrução, o sinal de escrita em memória, o endereço (apenas em `lw`, `lbu`, `sw` e `sb` que o ISS executou), o dado escrito e o fim do programa. Ao primeiro defeito de um programa, registra um `UVM_ERROR` e ignora o restante dele até o início do próximo, de modo que cada programa defeituoso gera uma única mensagem. Os registradores são verificados indiretamente: o epílogo de cada programa grava `x0` a `x29` na memória com `sw`, e o scoreboard confere cada palavra escrita.

Adaptação ao processador do colega. O mesmo código serve aos dois processadores por meio de um perfil escolhido por macro (`RV_PROFILE_MARINA`: 22 instruções, fim por `ecall`; `RV_PROFILE_COLEGA`: 19 instruções, fim por `jal x0,0`). Um adaptador no topo do testbench liga a interface do ambiente ao `cpu` do colega: lê, por referência hierárquica, `pc_cur`, `instr`, `mem_write`, `alu_y` e `rs2_val`, e carrega o programa e os dados em `u_imem.mem` e `u_dmem.mem` quando o driver pede.

8.3 Testes

Foram escritos três testes, cada um respondendo a uma pergunta (Tabela 9).

Tabela 9: Os três testes do ambiente.

Teste	Pergunta	Conteúdo
`riscv_basic_test`	O básico funciona?	4 programas escritos à mão: aritmética, laço, chamada de função, desvios
`riscv_random_test`	Funciona onde ninguém olhou?	N programas aleatórios (50 por padrão; `+NUM_PROGS=N`)
`riscv_illegal_test`	O que não devia executar é ignorado?	15 programas, um por instrução fora das 19

Programas aleatórios. Cada programa tem um prólogo (`lui` e `addi` dão valores conhecidos a `x1` até `x29`), um corpo de 20 a 100 instruções sorteadas do perfil do DUT e um epílogo (`sw` de `x0` a `x29` e a instrução de fim). Desvios e `jal` apontam só para frente, o que garante que todo programa termina. O `jalr` é gerado como o par `auipc` + `jalr`, às vezes com imediato ímpar, para exercitar o bit menos significativo do alvo.

Programas de instruções ilegais. As 15 candidatas são `sub`, `or`, `slli`, `lb`, `bltu`, `bgeu`, `ecall`, `ebreak`, `srai`, `sll`, `slt`, `mul`, `sh`, `lh` e `jalr` com `funct3` diferente de 0. Cada programa tem prólogo, a candidata, um marcador (`addi x6,x0,77`), o despejo dos registradores e o fim. Se o DUT executar a candidata, o trace diverge do ISS, que a trata como NOP.

8.4 Cobertura

O `riscv_coverage` recebe o trace esperado do ISS e amostra: o tipo de cada instrução do perfil (`cp_kind`); cada desvio tomado e não tomado (`cp_br`, `cp_taken`, `cx_branch`); as quantidades de deslocamento (`cp_shamt`); escrita em `x0` e em outros registradores

(`cp_rd_x0`); e o byte acessado dentro da palavra (`cp_byte_off`). Os quatro programas dirigidos alcançam 66,79% da cobertura; os programas aleatórios alcançam 100,00% em cada uma das cinco sementes testadas. O gerador aleatório, portanto, chega a cenários que a sequência escrita à mão não chega. Cobertura mede o que foi exercitado, não prova a ausência de defeitos: o achado da Seção 8.6 apareceu num teste dirigido pela especificação, não pela cobertura.

8.5 Execução e resultados no processador do colega

Cada teste é executado no xsim a partir da pasta `build` (comandos no Apêndice B). A Tabela 10 reúne os resultados.

Tabela 10: Resultados no processador do colega (xsim 2024.1).

Teste	Configuração	Programas	Ciclos	Resultado
`riscv_basic_test`	dirigido	4	53	PASSED
`riscv_random_test`	semente padrão	50	6 886	PASSED
`riscv_random_test`	5 sementes × 200 programas	1 000	138 347	PASSED
`riscv_illegal_test`	15 instruções fora do conjunto	15	950	FAILED (`mul`)

Em resumo: o processador do colega coincide com o ISS nos 4 programas dirigidos e nos 1 050 programas aleatórios, ou seja, em tudo o que implementa. A única divergência é a de uma instrução que deveria ser ignorada.

8.6 Achado: o `mul` é executado como `add`

O `riscv_illegal_test` acusou um único erro: no programa da instrução `mul`, o DUT escreveu o resultado de uma soma onde o ISS esperava nenhuma escrita. A causa está em `control.sv`. Para o `opcode` de instruções tipo R, o decodificador olha só o bit 30 da instrução (`funct7_5`), que distingue `add` de `sub`. O `mul` (`funct7` = `0000001`) tem esse bit em 0, passa pelo filtro e é decodificado pelo `funct3` como `add`.

```
OP_REG: begin
    alu_src = 1'b0;
    if (!funct7_5) begin                // sub is not supported
        case (funct3)
            F3_ADD: begin alu_op = ALU_ADD; reg_write = 1'b1; end
            F3_AND: begin alu_op = ALU_AND; reg_write = 1'b1; end
            F3_XOR: begin alu_op = ALU_XOR; reg_write = 1'b1; end
            default: ;
        endcase
```

```
    end
end
```

O defeito é mais geral do que o `mul`: qualquer instrução R com `funct7` diferente de 0 e bit 30 em 0, e `funct3` igual a `000`, `111` ou `100`, é executada como `add`, `and` ou `xor`. Isso foi observado também com `funct3` = `100` (`div`, que não está entre as 15 candidatas), executado como `xor`.

Confirmação por mutação. Para confirmar a causa, foi feita uma cópia do RTL do colega com uma única linha alterada em `cpu.sv`, de `assign funct7_5 = instr[30];` para `assign funct7_5 = |instr[31:25];`, de modo que qualquer `funct7` diferente de zero seja rejeitado. Um programa curto (`addi x1,x0,6`; `addi x2,x0,7`; `mul x3,x1,x2`; `sw x3,0(x0)`; e o equivalente com `div`) foi executado no RTL original e no corrigido, e os dados escritos foram comparados com o ISS (Tabela 11). Com a correção, o `mul` e o `div` passam a ser ignorados, e 60 programas aleatórios válidos continuam sem divergência.

Tabela 11: Dado escrito por `sw` após `mul` e `div` com $x1 = 6$ e $x2 = 7$.

Caso	Esperado (ISS)	Obtido	Interpretação
RTL original, `mul`	0	13	$6 + 7$: executado como `add`
RTL original, `div`	0	1	$6 \oplus 7$: executado como `xor`
RTL corrigido, `mul`	0	0	ignorado, como prometido no README
RTL corrigido, `div`	0	0	ignorado, como prometido no README

Limite desta confirmação: a mutação foi simulada com o Icarus Verilog e o mesmo ISS, não com o UVM no xsim, porque o xsim não estava disponível onde o experimento foi feito. A repetição do `riscv_illegal_test` no xsim com o `cpu.sv` corrigido, em que se espera `PASSED`, está descrita no Apêndice B.

8.7 Validação do próprio ambiente

Antes de confiar nos resultados acima, o ambiente foi aplicado ao processador desta equipe (Frente 1). O teste dirigido passou em 52 ciclos, e o aleatório passou com 50 programas, 6 926 ciclos e 100% de cobertura. Para verificar que o ambiente detecta defeitos, foi inserida uma mutação (`bge` trocado por `bltu`) no processador desta equipe: o ambiente a acusou; ao desfazê-la, o resultado voltou a PASSED (50 programas, 0 erros, 6 926 ciclos).

Duas falhas eram do próprio ambiente, e a elas se deve o resultado ter ficado confiável:

- **PASSED sem verificar nada.** Na primeira execução no processador do colega, o teste dirigido passou, mas cada programa rodou 300 ciclos e terminou por tempo esgotado: a sequência dirigida terminava os programas com `ecall`, que o colega não implementa. Passou a usar a instrução de fim do perfil (`jal x0,0`). Um PASSED só vale se o programa chegou ao fim.
- **Falsos positivos no scoreboard.** Das quatro falhas do primeiro teste de instruções ilegais, três (`lb`, `lh`, `sh`) eram do scoreboard: ele comparava o endereço da ALU apenas pelo `opcode`, inclusive quando o ISS tratava a instrução como NOP e não calculava endereço. A comparação agora só ocorre quando o ISS executou o acesso à memória. Restou o `mul`, o defeito real.

8.8 Limitações da Frente 2

- O ISS foi escrito por esta equipe: se interpretar mal uma instrução, o erro pode estar nos dois lados. Para reduzir esse risco, as saídas do ISS foram comparadas com o RTL dos dois processadores em outro simulador (Icarus Verilog), em 150 programas aleatórios, sem divergência.
- A verificação cobre as instruções do perfil e a lista de 15 candidatas ilegais; não é exaustiva. As entradas de periféricos do colega (chaves e botões) ficam em zero no topo do testbench; essa parte do processador não é exercitada.
- A confirmação por mutação do achado foi feita em Icarus Verilog (Seção 8.6); falta repeti-la no xsim.

9 Conclusão e próximos passos

O processador de ciclo único do Capítulo 7 foi implementado em SystemVerilog, estendido para 22 instruções e validado em simulação. O teste de sanidade passou, e o Genius do PBL02 rodou sem alteração nos cinco cenários, com pontuação e sequência de cores iguais às esperadas. Os diagramas do Vivado mostram a mesma divisão em controller e datapath do livro. Na Frente 2, o ambiente UVM verificou o processador da outra equipe: coincidiu com o modelo de referência em 4 programas dirigidos e 1050 aleatórios, e o teste de instruções ilegais achou que o `mul` é executado como `add`. A correção de uma linha, simulada em Icarus Verilog, elimina o defeito.

9.1 Limitações

- A verificação é somente em simulação. LEDs, chaves, botões e relógio do Genius são modelos em RTL; não houve síntese, implementação ou teste na placa.
- O relógio do modelo avança 1 ms por ciclo de clock, para que os atrasos de 2 s do jogo custem cerca de 2000 ciclos. Os tempos em microssegundos das ondas, portanto, não são tempos reais de jogo.
- O ISS do ambiente UVM foi escrito por esta equipe; um erro de interpretação pode estar nos dois lados (Seção 8).
- O testbench verifica a sequência de cores e a pontuação em cinco cenários escolhidos; não é uma verificação exaustiva das instruções.

9.2 Próximos passos

- Repetir no xsim o `riscv_illegal_test` com o `cpu.sv` corrigido (esperado: `PASSED`) e enviar o relato do defeito à outra equipe.
- Publicar o ambiente UVM e este relatório no repositório do projeto.
- Ensaiar a apresentação de 10 minutos com a demonstração do ambiente UVM.

Referências

[1] HARRIS, S. L.; HARRIS, D. M. *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022. Capítulo 7: Microarquitetura.

[2] NELSONAFN. *verif-env-template*. Repositório GitHub. Disponível em: `https://github.com/nelsonafn/verif-env-template`. Estrutura de projeto usada no PBL03.

[3] RISC-V INTERNATIONAL. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Disponível em: `https://riscv.org/technical/specifications/`.

[4] PENELU. *RISC_V-PBL*. Repositório GitHub. Disponível em: `https://github.com/Penelu/RISC_V-PBL`. Processador verificado na Frente 2.

[5] CHIPVERIFY. *UVM Tutorial*. Disponível em: `https://chipverify.com/tutorials/uvm`.

[6] AMD. *Vivado Design Suite User Guide: Logic Simulation (UG900)*, versão 2024.1.

A Como reproduzir

Ambiente: Windows, Git Bash (MINGW64) e Vivado 2024.1 instalado em `C:\Xilinx \Vivado \2024.1`. Os comandos são executados na pasta do projeto (`pbl03`), um por vez.

Teste de sanidade do processador

```
source setup_env.sh
rm -rf build
cmake -S . -B build -G "Unix Makefiles" -DTOP_NAME=testbench -DTEST_NAME=sanity -
    DSEED=1 -DVIVADO_PARMS="--R"
make SHELL="$(which bash)" compile
make SHELL="$(which bash)" sim
cd build
cmd //c "xsim top_sim -R"
```

Genius no processador

```
source setup_env.sh
rm -rf build
cmake -S . -B build -G "Unix Makefiles" -DTOP_NAME=genius_tb -DTEST_NAME=sanity -
    DSEED=1 -DVIVADO_PARMS="--R"
cp genius.hex data.hex build/
make SHELL="$(which bash)" compile
make SHELL="$(which bash)" sim
cd build
cmd //c "xsim top_sim -R"
```

O resultado esperado é a linha `RESULTADO: 5 cenarios passaram, 0 falharam`. Para ver as ondas, o último comando passa a ser `cmd //c "xsim top_sim -gui"`; no console Tcl do Vivado, adicionam-se os sinais e roda-se a simulação:

```
add_wave -radix hex /genius_tb/dut/PC
add_wave /genius_tb/led_prev
add_wave /genius_tb/dpad_btn
add_wave -radix unsigned /genius_tb/score
add_wave /genius_tb/ecall
run all
```

Diagramas (elaborated design)

No console Tcl de um Vivado aberto, um comando por vez:

```
create_project pbl03_view ../pbl03_view -part xc7a35tcpg236-1 -force
add_files [glob ../pbl03/rtl/*.sv]
set_property top genius_soc [current_fileset]
set_param synth.elaboration.rodinMoreOptions {rt::set_parameter
    dissolveMemorySizeLimit 262144}
```

Depois, em *RTL ANALYSIS*, abrir *Open Elaborated Design* e *Schematic*. Para ver só o processador, usar `riscvsingle` como topo.

Gerar o programa Genius

O `genius.hex` e o `data.hex` são gerados pelo script `genius/build_genius.sh` (Apêndice C), que usa o montador e o ligador do RISC-V GNU Toolchain; os arquivos prontos já acompanham o projeto.

Observações do ambiente

O último `make sim` termina com o erro `Unsupported architecture: i686-pc`, do script de inicialização do Xilinx no Git Bash. A elaboração já foi concluída e o `xsim` é chamado à parte, por `cmd //c`. No Git Bash, colar várias linhas de uma vez pode inserir caracteres extras; os comandos funcionam digitados ou colados uma linha por vez.

B Como reproduzir a Frente 2 (UVM)

Ambiente: o mesmo da Frente 1. O ambiente UVM vem em dois pacotes: um para o processador do colega (`pbl03_uvm_colega_env`) e outro para o desta equipe (`pbl03_uvm_riscv_env`). Após extrair o pacote, os comandos são executados na pasta extraída, um por vez.

Compilar (processador do colega)

```
source setup_env.sh
cmake -S . -B build -G "Unix Makefiles" -DTOP_NAME=riscv_colega_tb_top -DTEST_NAME
    =riscv_basic_test
make SHELL="$(which bash)" elaborate
cd build
```

Rodar os testes (a partir de `build`)

```
cmd //c 'xsim top_sim -R -testplusarg "UVM_TESTNAME=riscv_basic_test"'
cmd //c 'xsim top_sim -R -testplusarg "UVM_TESTNAME=riscv_random_test" -
    testplusarg "NUM_PROGS=200" -sv_seed 1'
cmd //c 'xsim top_sim -R -testplusarg "UVM_TESTNAME=riscv_illegal_test"'
```

O comando `make sim` não funciona neste ambiente (`Unsupported architecture`); o `xsim` deve ser chamado de dentro de `build`. O resultado aparece no fim da saída: `SB_SUMMARY` e o banner `PASSED` ou `FAILED`.

Repetir a confirmação do defeito no xsim

Em uma cópia do RTL do colega, trocar em `cpu.sv` a linha `assign funct7_5 = instr[30];` por `assign funct7_5 = |instr[31:25];`, recompilar e rodar de novo o `riscv_illegal_test`. Resultado esperado: `PASSED`.

C Código do SoC de teste do Genius

Este apêndice reúne os arquivos novos criados para executar o Genius no processador. O restante do RTL (controller, datapath, ALU e demais módulos) está descrito na Seção 3.

rtl/genius_soc.sv

```
// ============================================================
// genius_soc.sv  SoC de simulacao para rodar o Genius no nucleo
//
//   riscvsingle (nucleo)  +  imem (genius.hex)  +  dmem (data.hex)
//   + genius_periph (LEDs, switches, D-Pad, RTC)
//
// Decodificacao de endereco: 0xF000_0xxx = periferico, resto = dmem.
// ============================================================
module genius_soc(
    input  logic        clk, reset,
    input  logic [4:0]  switches,
    input  logic [3:0]  dpad_btn,
    output logic [95:0] leds,
    output logic        ECall
);
    logic [31:0] PC, Instr, ReadData, ALUResult, WriteData;
    logic        MemWrite;

    logic        io_sel;
    logic [31:0] io_rdata, mem_rdata;

    riscvsingle rv(
        .clk(clk), .reset(reset),
        .PC(PC), .Instr(Instr),
        .MemWrite(MemWrite),
```

```
        .ALUResult(ALUResult), .WriteData(WriteData),
        .ReadData(ReadData),
        .ECall(ECall)
    );

    imem #(.PROG_FILE("genius.hex")) imem1(.a(PC), .rd(Instr));

    dmem #(.DATA_FILE("data.hex")) dmem1(
        .clk(clk), .we(MemWrite & ~io_sel), .funct3(Instr[14:12]),
        .a(ALUResult), .wd(WriteData), .rd(mem_rdata)
    );

    genius_periph pp(
        .clk(clk), .reset(reset),
        .we(MemWrite),
        .is_load(Instr[6:0] == 7'b0000011),
        .addr(ALUResult), .wdata(WriteData),
        .sel(io_sel), .rdata(io_rdata),
        .switches(switches), .dpad_btn(dpad_btn), .leds(leds)
    );

    assign ReadData = io_sel ? io_rdata : mem_rdata;
endmodule
```

rtl/genius_periph.sv

```
// ============================================================
// genius_periph.sv  Perifericos mapeados em memoria (modelo de simulacao)
//
// Reproduz, para o nosso nucleo, os 4 perifericos que o Ripes oferecia
// ao programa Genius do PBL02. Regiao de I/O: enderecos 0xF000_0xxx.
//
//   0xF0000000 + 4*i  LED Matrix   (i=0..3)  escrita: cor RGB de 24 bits
//   0xF0000100        Switches                leitura: bits [4:0]
//   0xF0000200 + 4*d  D-Pad (d=0 UP,1 DOWN,2 LEFT,3 RIGHT)
//                                             leitura: 1 se houve clique
//                                             desde a ultima leitura
//                                             (captura de borda; a leitura limpa)
//   0xF0000300        RTC                     leitura: tempo em "ns"
//
// RTC_STEP: quanto o RTC avanca por ciclo de clock. Em hardware real o
// RTC avanca em tempo real; em simulacao usamos 1.000.000 ns/ciclo
// (1 ms por clock) para que um delay de 2 s custe so ~2000 ciclos.
// ============================================================
module genius_periph #(
    parameter RTC_STEP = 1000000
)(
    input  logic        clk, reset,
    // interface com o barramento de dados do processador
    input  logic        we,        // MemWrite
    input  logic        is_load,   // instrucao corrente e um load
    input  logic [31:0] addr,      // ALUResult
    input  logic [31:0] wdata,
    output logic        sel,       // endereco cai na regiao de I/O
    output logic [31:0] rdata,
    // "mundo externo" (controlado pelo testbench)
```

```
    input  logic [4:0]  switches,
    input  logic [3:0]  dpad_btn,  // pulso de 1 ciclo = clique
    output logic [95:0] leds       // 4 LEDs x 24 bits (LED i = leds[24*i +: 24])
);
    logic [3:0]  latch;
    logic [31:0] rtc;

    assign sel = (addr[31:28] == 4'hF);

    wire [3:0] page = addr[11:8];
    wire [1:0] idx  = addr[3:2];

    // leitura combinacional
    always_comb begin
        rdata = 32'b0;
        case (page)
            4'h1:    rdata = {27'b0, switches};
            4'h2:    rdata = {31'b0, latch[idx]};
            4'h3:    rdata = rtc;
            default: rdata = 32'b0;
        endcase
    end

    always_ff @(posedge clk) begin
        if (reset) begin
            latch <= 4'b0;
            rtc   <= 32'b0;
            leds  <= 96'b0;
        end else begin
            rtc <= rtc + RTC_STEP;

            // escrita nos LEDs
            if (we && sel && page == 4'h0)
                leds[24*idx +: 24] <= wdata[23:0];

            // D-Pad: o clique seta o latch; a leitura (lw) limpa
            for (int i = 0; i < 4; i++) begin
                if (dpad_btn[i])
                    latch[i] <= 1'b1;
                else if (is_load && sel && page == 4'h2 && idx == i)
                    latch[i] <= 1'b0;
            end
        end
    end
endmodule
```

genius/crt0.s

```
# crt0.s  inicializacao minima (o Ripes faz isso sozinho; no nosso SoC
# precisamos setar o ponteiro de pilha antes de entrar no main).
# sp = 0x3FF0 (topo da memoria de dados de 16 KB, alinhado)
.text
.globl _start
_start:
    lui  sp, 0x4        # sp = 0x00004000
    addi sp, sp, -16    # sp = 0x00003FF0
```

```
# (cai direto no main, que e a primeira instrucao do genius_pbl02.s)
```

genius/build_genius.sh

```
#!/bin/bash
# Monta o Genius para o nosso nucleo (so precisa rodar se alterar o .s)
set -e
AS="riscv64-unknown-elf-as -march=rv32i -mabi=ilp32 -mno-relax"
SYM="--defsym LED_MATRIX_0_BASE=0xF0000000 --defsym SWITCHES_0_BASE=0xF0000100 --defsym D_PAD_0_UP=0
     xF0000200 --defsym RTC_0_BASE=0xF0000300"
$AS crt0.s -o crt0.o
$AS $SYM genius_pbl02.s -o genius.o
riscv64-unknown-elf-ld -m elf32lriscv -Ttext=0x0 -Tdata=0x2000 -o genius.elf crt0.o genius.o
riscv64-unknown-elf-objcopy -O binary -j .text genius.elf text.bin
riscv64-unknown-elf-objcopy -O binary -j .data genius.elf data.bin
python3 -I make_hex.py
riscv64-unknown-elf-objdump -d -M no-aliases genius.elf > genius.dis
riscv64-unknown-elf-nm genius.elf | sort > genius.sym
```